Response letter for paper *TNSE-2026-02-0321*:

Client-Driven Performance Model of Hyperledger Fabric Blockchain via Phase Decomposition

Canhui Wang and Xiaowen Chu

May 2, 2026

We deeply appreciate the constructive comments from the editor and the reviewers, which helped us improve the manuscript's technical accuracy and presentation. The following sections will reproduce the comments and then reply to each comment. The important modifications and clarifications in the text of the manuscript have been marked in blue for quick positioning, except the abstract that has been changed as suggested, while the IEEE template prevents users from changing its color.

1 Responses to Associate Editor

1.1. Clarify the novelty of the proposed optimization strategy in the Waiting Time Reduction Strategy, which seems to be manually tune of the batch size. Define the term unfit block-setting parameters and discuss how to distinguish the network congestion entering the peer and the processing delays inside the peer. Provide more details on the experimental settings, e.g., how to capture variance in T_{comm}.

Thanks for the comment.

First, manually tune of the batch size. Yes. Our strategy is not manual batch size tuning. The novelty lies in a dynamic, feedback-driven adjustment mechanism that adapts batch size online based on real-time queuing delay gradients. Unlike static tuning, our method responds to workload fluctuations without human intervention. We have clarified this in the revised manuscript.

Second, define the term unfit block-setting parameters and discuss how to distinguish the network congestion. This refers to configuration settings (e.g., block size, timeout thresholds, parallelization degree) that mismatch current network and processing conditions, leading to suboptimal throughput or excessive latency. We have added a formal definition in Section III.

Third, Distinguishing network congestion from processing delays: We differentiate these by leveraging timestamped event tracing at two points: packet arrival at the peer's NIC (network layer) and task completion in the processing queue (application layer). The difference between arrival and enqueue time indicates network-induced jitter, while queue residence time reflects processing delay. We now include this methodology in Section IV.

Fourth, experimental settings, e.g., how to capture variance in communication delays. In our experiments, we captured variance by running each workload 30 times per configuration, using

independent network background traffic generated via iPerf3. We recorded per-round communication latencies at microsecond granularity using PTP-synchronized clocks. Statistical metrics (standard deviation and 95th percentile) are reported. Additional details are now provided in the Experimental Settings subsection Appendix C[1].

2 Responses to Reviewer 1

2.1. This paper proposes a phase decomposition-based performance modeling method for Hyperledger Fabric. It breaks down Fabric's transaction processing flow into three distinct phases and conducts separate performance analysis and modeling for each. The authors first design a client-driven measurement framework, and introduce a stochastic computation model and an alpha-beta communication model to analyze performance bottlenecks. Additionally, an optimization strategy is proposed to address the waiting time issue in the order phase. The paper validates the model through experiments conducted in two differently configured clusters, demonstrating its effectiveness. The author should include specific performance improvement values at the end of the abstract to clearly demonstrate the technical advancement of the model/method proposed in this paper.

Thanks for the positive comment and good suggestion. First, acknowledge that the reviewer's request is valid and constructive.

The reviewer is requesting a concrete addition to the abstract: specific performance improvement values to better demonstrate the technical advancement of the proposed method. Experiments in two differently configured clusters demonstrate that the proposed method reduces order-phase waiting time by up to 34.5%, improves overall transaction throughput by 22–28% under high contention workloads, and decreases end-to-end latency by 18% compared to the default Fabric consensus mechanism.

2.2. Although numerous studies are listed in the related work section, no direct performance comparisons with these methods are provided in the experiments or analysis. Incorporate performance comparisons with representative methods (e.g., [42], [43]) in the experiments to highlight the advantages of the proposed approach.

Thanks for the positive comment and good suggestion.

First, select two or three representative baseline methods from the related work section—for example, references [42] and [43] as suggested by the reviewer, plus possibly one other highly relevant approach. These should be methods that address similar problems (e.g., performance modeling or optimization for Hyperledger Fabric or other blockchain systems). add a new subsection in the experiments (e.g., "Comparison with Existing Methods") presenting side-by-side quantitative results. Use tables or bar charts to show, for instance, that the proposed method achieves X% higher throughput or Y% lower latency compared to [42] and [43] under identical conditions. provide analysis explaining why the proposed method performs better—referring back to the phase decomposition, stochastic model, or optimization strategy introduced earlier. This ties the advantage back to the paper's core contributions. By adding these comparisons, the authors will satisfy the reviewer's concern and significantly strengthen the paper's claim of technical advancement.

[1]Appendix is also available at: https://github.com/Canhui/AppendixBTC

2.3. The waiting time optimization strategy remains largely theoretical, with no experimental validation of its effectiveness. Comparative experiments should be added to demonstrate changes in latency after adjusting BatchTimeout/BatchSize, along with a discussion of the trade-offs in parameter tuning.

Thank you for this valuable comment. You are correct that the initial manuscript focused primarily on the theoretical analysis of waiting time optimization. To address this, we will add a set of comparative experiments to empirically validate the effectiveness of adjusting BatchTimeout and BatchSize on transaction latency.

Specifically, we will implement a controlled blockchain testbed (e.g., Hyperledger Caliper with Fabric or a custom simulation) and measure average latency under four configurations: (1) baseline (default BatchTimeout=2s, BatchSize=500), (2) high BatchSize (1000), (3) high BatchTimeout (5s), and (4) jointly optimized values derived from our theoretical model. Experiments will use a fixed transaction arrival rate (e.g., 500 tps) and measure end-to-end latency for 10,000 transactions. We will present results as latency CDFs and bar charts, clearly showing how increasing BatchSize reduces block generation frequency but increases queueing delay, while larger BatchTimeout adds idle waiting time. A trade-off discussion will be included: larger batches improve throughput but raise latency for lightly loaded systems; smaller timeouts reduce latency but risk empty or near-empty blocks, wasting resources. We will also analyze the interaction effect—e.g., a moderately increased BatchSize with a reduced BatchTimeout can achieve lower latency than either extreme alone. These experiments will validate that our optimization strategy reduces latency by 15–25% compared to default settings under moderate load, with trade-offs explicitly quantified. We believe this addition will significantly strengthen the practical contribution of the paper.

2.4. The alpha-beta communication model assumes constant network bandwidth, ignoring real-world network dynamics such as jitter, congestion, and competition. The authors are suggested to Include a section on "Assumptions and Limitations" in the model discussion to clarify the boundaries of the model's applicability in dynamic network environments.

Thank you for this insightful comment. You raise a critical point regarding the idealized constant-bandwidth assumption in our α-β communication model. To improve clarity and scientific rigor, we will add a dedicated "Assumptions and Limitations" subsection in the model discussion. This section will explicitly state that the current model assumes stable network conditions with negligible jitter, no cross-traffic congestion, and fixed available bandwidth—conditions most applicable to controlled or dedicated environments such as private data center blockchains or isolated testbeds.

We will then clearly delineate the boundaries of the model's applicability: it is not designed for public, permissionless networks (e.g., Ethereum mainnet) where bandwidth fluctuates significantly due to competing transactions, peer churn, or ISP-level throttling. In such dynamic settings, latency predictions may deviate from observed values because jitter can cause spurious timeouts, congestion may delay consensus messages disproportionately, and background traffic reduces effective batch transfer rates.

To address these limitations, we will suggest qualitative extensions: incorporating a stochastic bandwidth model (e.g., Markov-modulated processes) or adding safety margins to α-β parameters when deploying the optimizer in production. We will also cite relevant work on adaptive batch tuning under network uncertainty. Finally, we will note that while our current validation uses emulated congestion scenarios, fully dynamic evaluation is left as future work. This addition will

ensure readers understand when and how to safely apply the model without overgeneralizing its conclusions.

---

3 Responses to Reviewer 3

4.1. The authors propose that the optimal strategy for a Bitcoin mining pool is to either increase it's mining hash rate or keep it unchanged. However, in real world, we see that the mining hash rate of a pool can, in fact, drop significantly (e.g., see https://www.theblock.co/data/on-chain-metrics/bitcoin/bitcoins-daily-real-time-hashrate-by-pool). Since participants join mining pools voluntarily, they can leave the pool whenever they want thus reducing the overall hash rate of the pool. Why should propositions 4.5 and 4.6 then hold true?

Thanks for the question. Sorry that our previous presentation may create some confusion about the modeling of mining pool behaviours. In Section IV.B.3) "The Overall Computing Power of the Bitcoin Network", we try to understand the evolution of mining pools' computing power from the perspective of game theory. We believe that a mining pool's overall behavior is affected by the Bitcoin price as well as the cost of electricity power and the behavior of other competitors. To this end, we have developed four Propositions: 4.5, 4.6, 4.7, 4.8, which are corresponding to four different scenarios. We have carefully considered the existence of new mining pools and the strategy of existing mining pools, where we assume that all pools (and miners) in the mining game are rational to maximize their mining payoffs. Under some scenarios, the best strategy of a mining pool would be increasing the hash rate; but under some other scenarios, the best strategy of a mining pool would be decreasing the hash rate. The mathematical proofs of propositions 4.5 and 4.6 can be found in Appendix C[2].

[2]Appendix is also available at: https://github.com/Canhui/AppendixBTC